

DASE7506 MP1 Report

RoPE Language Model with Causal Repetition and Extended Training

Abstract

This project improves the supplied four-layer GPT while preserving the fixed WikiText-2 data, BPE-2048 tokenizer, evaluator, and causal window protocol. The final predictor replaces learned positional embeddings with rotary position embedding, extends optimization from a freshly trained 1,200-step parent by 12,000 updates, averages the later neural weights, and adds a validation-selected distribution over continuations of repeated suffixes inside the current input window. The frozen model achieves 1.632893777909 full-test bits per byte, compared with 2.101260438087 for the supplied baseline. It remains within all evaluation limits: 2.244 times the baseline CPU scoring time, 1.577 GiB peak RAM, and 4.051 MiB of inference assets. Equal-budget and same-weight controls show that both RoPE and causal repetition contribute, while the larger training budget remains an additional source of improvement.

1 Task and Evaluation Metric

The model predicts the next token for independent windows containing at most 256 targets. At position t , it may use only the observed prefix through t . The evaluator scores every target except the first token of each split exactly once, including the final short window, and resets model state between windows. Development and model selection use the validation split. The test split is evaluated only after the method and checkpoint are frozen.

The ranked metric is bits per byte rather than token perplexity. If NLL is the summed negative natural-log probability of all scored next tokens and B is the number of bytes in the raw UTF-8 text, then

$$\text{BPB} = \text{NLL} / (B \ln 2)$$

Dividing by raw bytes makes the score tied to the original text rather than the token count. Lower BPB means that the model assigns higher probability to the observed text.

2 Baseline Model

The supplied baseline is a decoder-only GPT with four Transformer blocks, model width 128, four attention heads, GELU feed-forward layers, pre-normalization, residual connections, and a context length of 256. It uses learned token and absolute position embeddings. The token embedding and output projection share weights. With vocabulary size 2,048, the baseline has 1,088,256 trainable parameters. AdamW trains the model on random 257-token segments so that each update predicts 256 targets per sequence.

3 Proposed Model

3.1 Rotary Position Embedding

The first change removes the learned absolute position table and applies rotary position embedding (RoPE) to the query and key vectors in every attention head. For each pair of feature dimensions, position m rotates the vector by an angle m times a fixed frequency. For a two-dimensional pair (x_1, x_2) , the operation is

$$R(m, \theta)[x_1, x_2] = [x_1 \cos(m\theta) - x_2 \sin(m\theta), x_1 \sin(m\theta) + x_2 \cos(m\theta)]$$

Applying the same construction at several frequencies lets the query-key dot product depend directly on relative displacement. Values are not rotated, and scaled dot-product attention remains strictly causal. Removing the absolute position table reduces the parameter count to 1,055,488. The intended benefit is a more direct representation of token distance without increasing inference assets.

3.2 Causal Repetition Distribution

The final predictor also uses repetitions that have already appeared within the current input window. To predict after position t , it searches earlier positions $j < t$ whose suffix matches the suffix ending at t . It first checks a two-token suffix and falls back to a one-token match when no longer match exists. Each matching position contributes its continuation token $x(j + 1)$ to a histogram q_{copy} over the vocabulary.

This lookup is causal. Since $j < t$, the continuation at $j + 1$ is no later than t and is therefore already part of the observed prefix. The histogram is built separately for every example and every forward call; it stores no validation or test answers and carries no state across windows.

3.3 Probability Mixture

Let p_{neural} be the neural softmax distribution and n the number of matched continuations. The final predictor uses

$$w = \alpha n / (n + s), p_{\text{final}} = (1 - w)p_{\text{neural}} + wq_{\text{copy}}$$

Validation selected $\alpha = 0.25$, maximum suffix order 2, and smoothing $s = 2$. The weight grows when several earlier matches agree, but remains below α . When no match exists, $n = 0$ and the method reduces exactly to the neural model. The neural component always has positive weight, so the returned log probabilities remain finite.

4 Development and Training Process

4.1 Equal Budget Architecture Experiment

The first development round isolated the positional change. The official baseline and the RoPE model used the same seed, batch size, context length, optimizer recipe, and 1,200 updates. Each processed 9,830,400 training targets. RoPE reduced validation BPB from 2.0710836385 to 1.9223254395. This result justified keeping RoPE for the later training stage. It does not prove

that every difference comes only from position encoding, because deleting the position table changes parameter count and random-number consumption.

4.2 Extended Optimization and Weight Averaging

A new 1,200-step RoPE parent was trained from random initialization with seed 17. A fresh AdamW optimizer then continued from its weights for 12,000 updates with seed 23, batch size 32, peak learning rate 0.0006, weight decay 0.1, 100-step warmup, cosine decay, and gradient clipping at 1. Training and validation used CPU FP32 with four threads. The parent processed 9,830,400 targets and continuation processed 98,304,000, giving 108,134,400 targets in the final ancestry.

Full validation was run every 600 continuation updates. From update 6,000 onward, one neural-weight snapshot was added to a running average every 100 updates. Averaging changes neither architecture nor inference cost. It reduced the final validation BPB from 1.6708548535 for the live weights to 1.6615916362 for the average of 61 snapshots, so the averaged checkpoint at update 12,000 was selected.

4.3 Validation Selection of Repetition

After freezing the selected neural weights, a fixed ten-point validation grid compared repetition disabled with nine combinations of α in $\{0.10, 0.25, 0.40\}$ and maximum suffix order in $\{1, 2, 3\}$; smoothing remained 2. This search did not update the neural parameters. The selected $\alpha = 0.25$ and order 2 achieved 1.6186115943 validation BPB. The nearest alternatives were order 3 at 1.6188149774 and order 1 at 1.6193124481, showing that the choice was useful but not based on a large separation among the best candidates.

5 Comparison and Ablation

Predictor	Training targets	Validation BPB	Purpose
GPT baseline	9,830,400	2.0710836385	Initial reference
RoPE parent	9,830,400	1.9223254395	Equal-budget comparison
Final neural weights, repetition off	108,134,400	1.6615916362	Mechanism ablation
Same weights, repetition on	108,134,400	1.6186115943	Selected predictor

The equal-budget comparison attributes a 0.1487581990 validation-BPB reduction to the RoPE model and its associated parameterization under the short training recipe. The same-weight ablation isolates the repetition mechanism: enabling it lowers validation BPB by 0.0429800419 with identical neural weights and training ancestry. The remaining improvement of the final model also reflects its much larger optimization budget, so the full gain over the baseline cannot be assigned to either mechanism alone.

6 Final Results and Resource Use

Measure	Observed	Requirement
Full-test CPU FP32 BPB	1.632893777909	Lower than 2.101260438087 baseline
Median CPU scoring-time ratio	2.244 times	At most 5 times baseline
Conservative time ratio	2.286 times	Max student divided by min baseline
Peak process-tree RAM	1.577 GiB	At most 4 GiB
Uncompressed inference assets	4.051 MiB	At most 64 MiB

The frozen predictor produced the same test BPB in three complete runs and in a separate extracted directory using the same dependency environment. The test improvement over the baseline is 0.4683666602 BPB, or 22.3 percent relative to the baseline value. Repetition increases computation because it compares positions inside each window, explaining why scoring is about 2.24 times slower. The model remains comfortably below the CPU, RAM, and asset limits.

Nine unit and contract tests checked causality, normalization, example independence, state reset, gradients, target coverage, and repetition behavior. Additional tests loaded the final checkpoint and verified full-context future-token causality, agreement between a short prefix and the same prefix inside a longer window, deterministic repeated calls, finite normalized probabilities, and exact equality with the neural predictor when repetition is disabled. SHA-256 checks confirmed that `common.py`, `evaluate.py`, the data, tokenizer, baseline configuration, and official contract test were unchanged.

7 Reproduction

Windows, Python 3.12.10, PyTorch 2.7.1, NumPy 2.5.3, tokenizers 0.21.4. The checkpoint can be evaluated without retraining. From the code directory, install the pinned dependencies and run the following command:

```
python evaluate.py --checkpoint runs/retrain-selected-12000/checkpoint.pt --device cpu --precision fp32 --threads 4 --split test
```

Full installation and three-stage training commands are in the submitted README.md.

8 Critical analysis and Limitations

The experiments support three conclusions. First, RoPE improves the short-budget model under a matched training-target comparison. Second, averaging stabilizes the late continuation checkpoints at no inference cost. Third, causal repetition contributes an additional same-weight validation improvement and transfers to the frozen test result. The repetition benefit is plausible for encyclopedia text because entity names and local phrases often recur within a 256-token window.

The evidence also has limits. No baseline was trained for the same 108,134,400-target ancestry,

so the report does not claim that all final improvement would persist against a long-trained baseline. Only one initial seed and one continuation seed were used. The best three repetition configurations were close on validation, and the mechanism adds quadratic position comparisons inside each window. Timing and memory measurements are specific to this host, although they were collected by alternating baseline and student runs under the same settings.

An earlier model had already been evaluated on the test split before the 12,000-step continuation experiment was requested. For the new run, the schedule was specified in advance, all model and mixture selection used validation, and code plus checkpoint hashes were frozen before the new test evaluations. The final test score was then repeated only for reproduction and resource measurement.

9 AI Assistance Declaration

Codex and ChatGPT assisted with implementation, execution of the training and evaluation workflow, diagnosis of intermediate results, reproducibility checks. The recorded checks include repeated CPU FP32 evaluations, process-tree peak-RAM monitoring, inference-asset measurement, checkpoint contract tests, protected-file hash verification, and fresh-directory reproduction. I reviewed the full implementation, commands, and experimental records. The reported BPB, timing, and memory values come from local executions of the supplied evaluator; they were not estimated by the AI. The project uses only the supplied training data and fixed tokenizer and evaluation protocol, with no external pretrained weights or training text.

References

- [1] Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv.org. <https://arxiv.org/abs/2104.09864>
- [2] DASE7506 MP1 course package: baseline model, BPE-2048 tokenizer, WikiText-2 splits, fixed evaluator, and project guide.